

The Sphere Eversion Program

Concept: Bill Thurston
Program: Nathaniel Thurston
Documentation: Silvio Levy

March 5, 2022

Introduction

This document, by Silvio Levy, describes the C++ program **evert** for turning the sphere inside out without tears, creases, or pinches. The program was written by Nathaniel Thurston, and is based on ideas of Bill Thurston.

We assume you are familiar with the basic ideas of the algorithm for turning the sphere inside out described in the computer-animated video *Outside In*. This video, created at the Geometry Center (University of Minnesota) and directed by Silvio Levy, Tamara Munzner and Delle Maxwell, gives an elementary description of the eversion, and shows it step by step, from different points of view. You should also consult the full-color illustrated book *Making Waves: A Guide to the Ideas Behind Outside In*, which comes with the video. *Outside In* and *Making Waves* are distributed by

A K Peters
289 Linden Street
Wellesley MA 02181
phone 617-235-2210
fax 617-235-2404

You can also point your favorite Web browser to <http://geom.umn.edu/locate/oi>.

Comments and questions can be sent to Silvio Levy at levy@geom.umn.edu, or to Nathaniel Thurston at njt@math.berkeley.edu.

Preparation

Evert is written in C++, GNU dialect (gcc). Binary executables are provided for the Sparc, the SGI, and the Next; however, the program should compile and run on any system having gcc. If you have a different machine and want to compile the program for it, type “make” and see what happens. There may be some harmless warning messages at all events. If there is more serious trouble, send mail to levy@geom.umn.edu: I will try to help you, but no guarantees.

Evert takes no input, other than command-line options. Its output is a file of numbers representing the sphere (or, more usually, a piece of it) at a given point in the eversion.

This means that, in order to see the output, you need to be able to transform the numbers—that is, the geometry of the sphere—into a picture. There are various programs that do this. One is called Geomview, and is freely available from the Geometry Center, by anonymous ftp from the server geom.umn.edu, in directory pub/software. It runs on many computers under X-windows, as well as on SGI and Next workstations. Geomview reads in a geometry file like the one output by **evert**, and produces an image in a window, where it can be interactively manipulated: rotated, translated, colored, etc. Geomview can also rewrite the data so that it becomes acceptable to Renderman, a program that creates a static image of much higher quality than that produced by an interactive viewer. The images that appear in *Outside In* were created using Renderman. Renderman was created by Pixar and is sold by them.

If you don't have Geomview but have a different viewing program, you may be able to adapt the output of **evert** to serve as input to your program, either by changing the code or by writing an output filter. If you do that, we would appreciate hearing from you and getting a copy of your code.

Running evert

This is how you run **evert** from the command line:

```
evert [options] [> outfile]
```

Here > is the Unix character for sending output for a file; if you are running the program on another system, change accordingly.

The options are the following:

- time *t* (float)** Creates the geometry of the sphere at time *t*. Time varies from 0 to 1, during which period it parametrizes the whole eversion process. From $t = t_0 = 0$ to $t = t_1 = 0.1$, the sphere becomes corrugated. From t_1 to $t_2 = 0.23$, the poles are pushed through each other. From t_2 to $t_3 = 0.6$, the sphere gets twisted: the poles rotate in opposite directions, and the loop around the equator disappears. From t_3 to $t_4 = 0.93$, the strips and corrugations are pushed through the center, with the poles held fixed. Finally, from t_4 to $t = 1$, the sphere loses its corrugations. The values of t_0 , t_1 , t_2 , t_3 , and t_4 are not hardwired: they can be controlled by the next five options. They should obey the inequalities $0 \leq t_0 < t_1 < t_2 < t_3 < t_4 < 1$, or unpredictable results will be obtained.
- corr t_0 (float)** Set the milestone t_0 for the beginning of the corrugation phase; default 0 (see **-time**). No output is generated if $t < t_0$.
- push t_1 (float)** Set the milestone t_1 for the beginning of the pushing phase; default 0.1 (see **-time**).
- twist t_2 (float)** Set the milestone t_2 for the beginning of the twisting phase; default 0.23 (see **-time**).

- unpush t_3 (float)** Set the milestone t_3 for the beginning of the unpushing phase; default 0.6 (see **-time**).
- uncorr t_4 (float)** Set the milestone t_4 for the beginning of the uncorrugation phase; default 0.93 (see **-time**).
- nstrips n (float)** The sphere is made of n guide strips alternating with n corrugations. The default for n is eight. If you set n too low you will get pinch points, so the eversion will not be a regular homotopy.
- umin $u_{\min}$ (float)** The sphere is parametrized by the variable u in the north-south direction, that is, a fixed value of u gives a parallel of latitude. The parameter u ranges between $u_{\min}$ and $u_{\max}$. The default for $u_{\min}$ is 0, corresponding to a pole.
- umax $u_{\max}$ (float)** Maximum value of the parameter u (see previous entry); default is 1, indicating the equator. Setting $u_{\max} = 2$ (and keeping $u_{\min} = 0$) gives a strip from pole to pole; but there is a better way to do this, as explained in the Symmetries section below.
- du Δu (float)** Increment in the value of u . The default value is $u = \frac{1}{12}$, so the fundamental unit of the sphere is subdivided into twelve, from pole to equator.
- vmin $t_{\min}$ (float)** The sphere is parametrized by the variable v in the east-west direction, that is, a fixed value of v gives a meridian. The parameter v ranges between $v_{\min}$ and $v_{\max}$; when $v_{\max} = v_{\min} + 1$ the result is a “fundamental unit” of the sphere, from which the rest can be deduced by rotation, as explained in the Symmetries section below.
- vmax (float)** Maximum value of the parameter v (see previous entry); default is 1, which together with $v_{\min} = 0$ gives a fundamental unit of the sphere. Setting $v_{\max}$ to the number of strips (see **-nstrips**) and keeping $v_{\min} = 0$ gives all the strips, and so a whole hemisphere; but there is a better way to do this, as explained in the Symmetries section below.
- dv Δv (float)** Increment in the value of v . The default value is $v = \frac{1}{12}$, so the fundamental unit of the sphere is subdivided into twelve, from the western edge to the eastern.
- whole (boolean)** If this flag is set, the program produces both the fundamental unit of the sphere and a list of transformations which rotate copies of it to cover the entire sphere.
- parts *partnames*** If set, the program produces the fundamental unit of the sphere and a list of transformations which take it to the given parts of the sphere. Parts are named by a sign (+ or – for the $+Z$ or $-Z$ hemispheres) followed by a number between zero and one less than the number of strips. For example, if there are eight strips, **-whole** is equivalent to **-parts +0+1+2+3+4+5+6+7-0-1-2-3-4-5-6-7**. The default is equivalent to **-parts +0**. See also the Symmetries section below.

- scale *s* (float)** Scale the whole geometry up or down by a factor *s*.
- bezier (boolean)** If this flag is set, the program outputs bicubic Bézier patches rather than a quadrilateral mesh (the default). This allows some viewers (including Geomview) to do a better rendering job. When you choose Bézier patches you'll probably want to set the options **-du** and **-dv** to a largish value (say 0.2, for a 5×5 grid), since you won't need as fine a grid as the default 12×12 .

Symmetries

When you have one “fundamental unit” of the everting sphere, you can reconstruct the other by applying rotations. Suppose you ran the command

```
evert -bezier -du 0.25 -dv 0.25 -t 0.5 > evert.out
```

This catches the sphere in the middle of the twisting phase ($t = 0.5$), and produces a mesh of 4×4 Bézier patches representing one-sixteenth of the sphere (from pole to equator, 45° degrees of longitude). If you now type

```
geomview evert.out
```

you get to see this one-sixteenth. If you want to see more of the sphere, there are three possibilities. One is to rerun **evert** with **-umax 2** and **-vmax 8**, as explained under those options; this creates a very large output file representing the whole sphere. Another is to run **evert** with the **-whole** option, causing it to instantiate 16 other *instances* of the given piece; the instantiation is specified in terms of 4×4 transformation matrices. A third possibility is to use the **-parts** option to instantiate selected strips. Each strip has a name, beginning with + or - according to its hemisphere, and followed by a number between zero and *nstrips* - 1. Assuming *vmin* = 0 and *vmax* = 1 (the default), strip *+k* and *-k* share a common edge along the equator. The **-parts** option takes a sequence of these names (written with no intervening blanks). For example, **-parts +0+1+2+3-4-5-6-7** creates strips 0 through 3 in the northern and 4 through 7 in the southern hemisphere, yielding two antipodal quarter-spheres which are interchanged by the eversion; and **-parts +1-1+3-3+5-5+7-7** yields every alternate strip running from pole to pole.